02 — LLM Setup via SAP Gen AI Hub

What it is: the standard way to get a callable LLM object from SAP AI Core using the Gen AI Hub **proxy** — one model, one deployment, LangChain-compatible. Every higher-level template (query agent, RAG, chatbot) calls this first.

Code Pattern

```python
import os
from dotenv import load_dotenv
from gen_ai_hub.proxy.langchain.openai import ChatOpenAI       # LangChain-compatible wrapper
from gen_ai_hub.proxy.core.proxy_clients import get_proxy_client

load_dotenv()

def get_llm(model: str = "gpt-4o-mini", temperature: float = 0):
    deployment_id = os.getenv("LLM_DEPLOYMENT_ID")   # from AI Launchpad -> Deployments
    if not deployment_id:
        raise EnvironmentError("LLM_DEPLOYMENT_ID not set in .env")

    proxy_client = get_proxy_client("gen-ai-hub")     # resolves AICORE_* creds automatically

    llm = ChatOpenAI(
        proxy_model_name = model,        # logical model name, e.g. "gpt-4o-mini"
        proxy_client      = proxy_client,
        deployment_id     = deployment_id,  # WHICH deployment actually serves the model
        temperature        = temperature,    # 0 = deterministic, 1 = creative
    )
    return llm

llm = get_llm()
from langchain.schema import HumanMessage
response = llm.invoke([HumanMessage(content="Say hello in one sentence.")])
print(response.content)
```

Embedding Model Variant (same pattern, different class)

```python
from gen_ai_hub.proxy.langchain.openai import OpenAIEmbeddings

model = OpenAIEmbeddings(
    proxy_model_name = "text-embedding-ada-002",     # 1536 dimensions
    proxy_client      = get_proxy_client("gen-ai-hub"),
    deployment_id     = os.getenv("EMBEDDING_DEPLOYMENT_ID"),
)
vectors = model.embed_documents(["text one", "text two"])   # -> list[list[float]]
```

Key Points

- `deployment_id` != `model name` — the deployment is the *running instance* on AI Core; the model name tells the proxy which underlying model that deployment serves.
- `get_proxy_client("gen-ai-hub")` resolves credentials from env vars / `~/.aicore/config.json` / `VCAP_SERVICES` automatically — no manual token handling.
- Returned objects are **LangChain-native** (`ChatOpenAI`, `OpenAIEmbeddings`) — they work with any LangChain chain, prompt template, or agent unmodified.
- `temperature=0` for deterministic tasks (SQL generation, classification); higher for creative writing.
- Embedding dimension **must match** what your vector table column was created with (1536 for `ada-002`, 3072 for `text-embedding-3-large`).

Interview Q&A

Q: What does "harmonized API" mean in the Gen AI Hub context? A: Every supported provider (OpenAI, Google, Anthropic, etc.) is exposed through the same LangChain-compatible interface, so swap-

ping models is a config change, not a rewrite.

Q: Why is `deployment_id` required in addition to the model name? A: SAP AI Core requires you to explicitly deploy a model before it's callable — the deployment ID identifies *your* provisioned, billable instance of that model.

Q: When would you use the proxy instead of the Orchestration Service? A: When you only need a single model call with no masking/filtering/grounding pipeline — simpler and lower latency than orchestration for straightforward use cases.